

Optional Class in Java

Optional & Date/Time : Optional class patterns (avoid null);
java.time API

Rahul Guha || SME || Java FS Training || HCL
2. Modern Java (8–21)

Optional Class in Java

What is Optional?

- `Optional<T>` is a container object introduced in Java 8 that may or may not contain a non-null value. It helps developers represent the possible absence of a value and handle it explicitly instead of relying on repeated null checks.
- Package: `java.util.Optional`
- The `java.util.Optional<T>` class, introduced in Java 8, is a container object that may or may not contain a non-null value. It provides a type-level solution to explicitly represent the absence of a value, helping developers avoid verbose null checks and reducing the risk of encountering a `NullPointerException` (NPE). According to the Java Official Documentation, `Optional` is primarily intended for use as a method return type where there is a clear need to convey "no result".

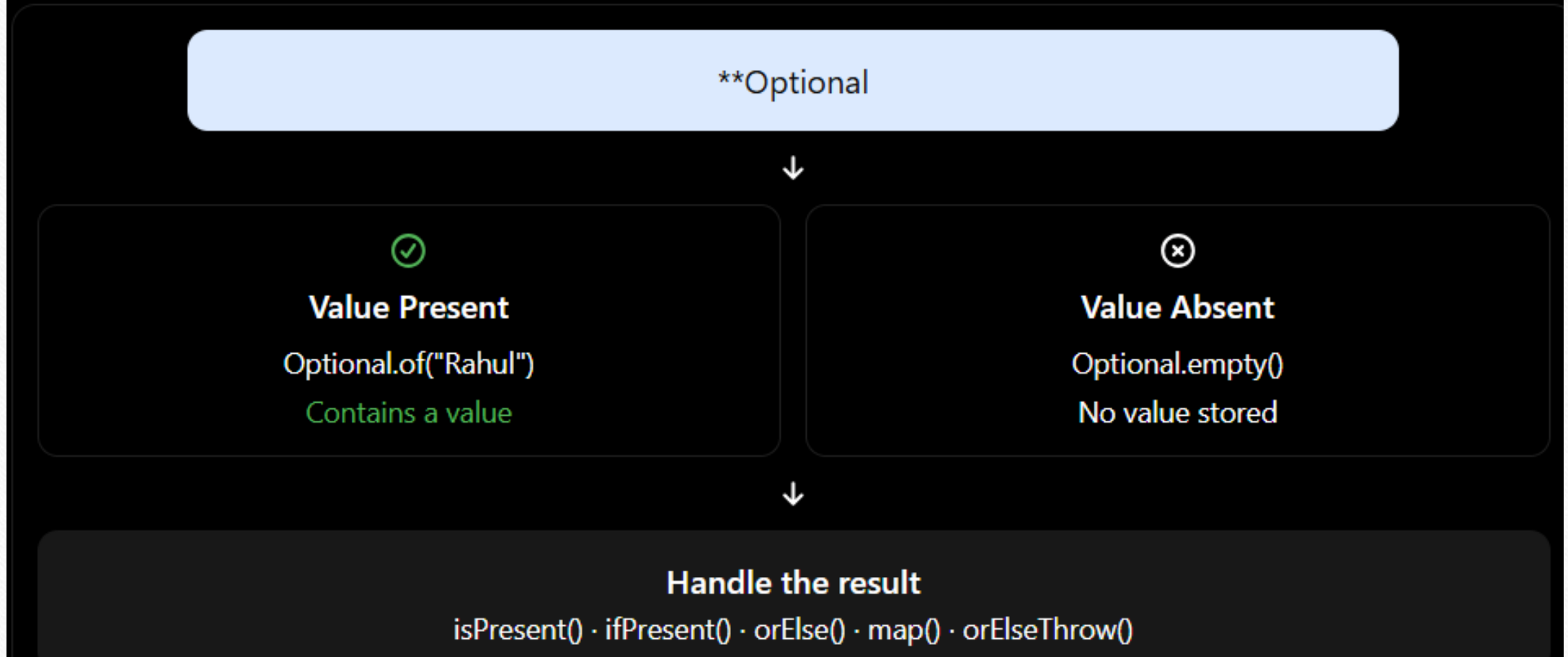
Optional Class in Java

Creating Optional Objects

The Optional class provides factory methods to create objects based on whether a value is available or may be absent.

- `Optional.empty()`: Creates an Optional that contains no value.
- `Optional.of(value)`: Creates an Optional with a non-null value. It throws `NullPointerException` if the value is null.
- `Optional.ofNullable(value)`: Creates an Optional containing the value if it is non-null; otherwise, returns an empty Optional.

2. Optional class diagram



Available Methods in Optional Class

- **Optional.isPresent()** - Checks if a value is present in the Optional.
- **Optional.get()** - Gets the value from the Optional if present, otherwise throws NoSuchElementException.
- **Optional.orElse(T other)** - Returns the value if present, otherwise returns the specified other value.
- **Optional.orElseGet(Supplier other)** - Returns the value if present, otherwise returns the result produced by the supplying function.
- **Optional.orElseThrow(Supplier exceptionSupplier)** - Returns the value if present, otherwise throws the exception produced by the supplying function.
- **Optional.ifPresent(Consumer consumer)** - Executes the specified consumer if a value is present.
- **Optional.filter(Predicate predicate)** - Filters the value of the Optional if present based on the specified predicate.
- **Optional.map(Function mapper)** - Maps the value of the Optional if present using the specified mapper function.
- **Optional.flatMap(Function> mapper)** - Maps the value of the Optional if present to another Optional using the specified mapper function.

Common Methods in Optional

Method	Description
<code>empty()</code>	Returns an empty Optional instance.
<code>of(T value)</code>	Returns an Optional containing the specified non-null value.
<code>ofNullable(T value)</code>	Returns an Optional containing the specified value if non-null; otherwise, returns an empty Optional.
<code>equals(Object obj)</code>	Checks whether another object is equal to this Optional.
<code>filter(Predicate<? super T> predicate)</code>	Returns the Optional if its value matches the given predicate; otherwise, returns an empty Optional.
<code>flatMap(Function<? super T, Optional<U>> mapper)</code>	Applies the given mapping function if a value is present and returns its Optional result; otherwise, returns an empty Optional.
<code>get()</code>	Returns the value if present; otherwise, throws <code>NoSuchElementException</code> .
<code>hashCode()</code>	Returns the hash code of the value if present; otherwise, returns 0.
<code>ifPresent(Consumer<? super T> consumer)</code>	Performs the given action if a value is present; otherwise, does nothing.
<code>map(Function<? super T, ? extends U> mapper)</code>	Applies the given mapping function if a value is present and returns an Optional containing the mapped result if non-null.
<code>orElse(T other)</code>	Returns the value if present; otherwise, returns the specified default value.
<code>orElseGet(Supplier<? extends T> other)</code>	Returns the value if present; otherwise, invokes the supplier and returns its result.
<code>orElseThrow(Supplier<? extends X> exceptionSupplier)</code>	Returns the value if present; otherwise, throws the exception supplied by the given supplier.
<code>toString()</code>	Returns a string representation of the Optional, suitable for debugging.